

DEPI Graduation ProjectDEPI Graduation Project

Wa3eny

Cybersecurity Awareness Platform

Project Documentation Report

Prepared from the project README and repository structure of the Aegis AI graduation project.

Document Field	Value
Project name	Wa3eny
Project type	AI-powered cybersecurity awareness and phishing-detection training platform
Primary stack	FastAPI, Supabase (PostgreSQL + Auth), Anthropic Claude API, HTML/CSS/JavaScript
Document version	1.0

Contents

Section	Topic
1	Executive Summary
2	Project Planning and Management
3	Literature Review and Evaluation Framework
4	Requirements Gathering
5	System Analysis and Design
6	Implementation and Execution
7	API Documentation
8	Testing and Quality Assurance
9	User Manual
10	Future Enhancements
Appendix A	Project File Map
Appendix B	Guideline Compliance Matrix

1. Executive Summary

Aegis AI is a web-based cybersecurity awareness platform that helps organizations and individuals recognize phishing attempts through realistic, AI-generated training scenarios, instant feedback, risk scoring, and gamified learning. The platform combines a FastAPI backend, a Supabase PostgreSQL data layer with Supabase Auth, an Anthropic Claude scenario-generation service, and a lightweight HTML/CSS/JavaScript frontend.

The project addresses a common security problem: many attacks succeed because users are pressured into clicking links, opening suspicious attachments, sharing verification codes, or trusting forged identities. Aegis AI turns those risks into repeatable practice sessions tailored to company departments, individual users, and senior citizens.

Project outcome: The completed documentation package explains the project structure, planning approach, requirements, architecture, database model, API surface, testing approach, and the user workflows required by the project documentation guidelines.

Capability	Description
AI scenario generation	Uses the Anthropic Claude API to create phishing and safe-message simulations across email, WhatsApp, and login-page contexts.
Organization mode	Supports department-specific training for Accounting, HR, IT, Marketing, and Sales.
Individual mode	Provides personal phishing practice across banking, shopping, and social media contexts.
Senior awareness mode	Offers simplified awareness guidance and quiz-based learning for older users.
Risk dashboard	Summarizes department attempts, average scores, correct classifications, and risk levels.
Leaderboard	Encourages continued participation through points and badges.

2. Project Planning and Management

2.1 Project Proposal

The proposed project is an AI-assisted training platform for phishing awareness. Its objective is to improve user decision-making before harmful actions occur, especially when users receive urgent messages, suspicious links, credential requests, or fraudulent offers.

Area	Project definition
Problem	Users and employees often struggle to identify social-engineering attacks under time pressure.
Objective	Provide realistic, repeatable phishing simulations with feedback that teaches the user what to check.
Scope	Web-based training, AI-generated scenarios, user scoring, department risk measurement, and setup documentation.
Audience	Organizations, employees, individual learners, senior citizens, instructors, and project evaluators.
Success measure	Users complete scenarios, identify red flags, and improve average awareness scores over time.

2.2 Project Plan and Milestones

The timeline below follows standard graduation-project guideline categories and adapts them to the Aegis AI deliverables.

Milestone	Deliverable
Project Planning and Management	Proposal, objectives, project scope, KPIs, roles, and risks.
Literature Review	Evaluation context, improvement areas, and grading criteria alignment.
Requirements Gathering	Stakeholders, user stories, functional and non-functional requirements.
System Analysis and Design	Architecture, database model, flows, API design, UI/UX guidelines.
Implementation	Source code, setup instructions, database schema, and execution guide.
Final Presentation, Testing, and Reports	Test plan, user manual, technical documentation, presentation flow.

2.3 Task Assignment and Roles

Role	Responsibilities
Project lead	Coordinate milestones, maintain scope, review deliverables, and prepare final submission.
Backend developer	Build FastAPI routers, service layer, request/response models, and environment configuration.
Frontend developer	Implement the single-page interface, training flows, dashboard, and API integration.
Database designer	Create Supabase schema, relationships, indexes, row-level security policies, and seed data.
AI integration owner	Design prompts for the Claude API, validate generated scenario structure, and handle provider errors.
QA/documentation owner	Create test cases, run verification, prepare setup docs, and maintain the user manual.

2.4 Risk Assessment and Mitigation Plan

Risk	Impact	Mitigation
Missing API keys	AI generation or database access may fail during demo.	Prepare .env early, document setup, and verify keys before presentation.
Claude API rate/quota limits	Scenario generation may slow down or stop.	Monitor usage and prepare sample scenarios for backup testing.
Weak production security	Secrets, CORS, or access policies could expose data.	Use environment variables, restrict CORS, enable RLS, and rotate keys.
Low user adoption	Training effect may be limited.	Use gamification, department-specific scenarios, and clear feedback.
Incomplete test coverage	Regressions could go unnoticed.	Keep automated API tests and add integration tests as the project grows.
Deployment mismatch	Local behavior may differ from production.	Document environment variables and health checks for each environment.

2.5 Key Performance Indicators

KPI	Target / measurement
Scenario completion	Number of completed training attempts by user, department, and organization.
Classification accuracy	Percentage of scenarios correctly identified as phishing or safe.
Average awareness score	Average scenario score based on classification accuracy.
Department risk level	Risk indicator derived from average score, attempts, and correct classifications.
Leaderboard engagement	Point totals, badge thresholds, and repeat participation.
System availability	Healthy backend response from /health and reliable frontend loading.

3. Literature Review and Evaluation Framework

The project is motivated by a human-centered cybersecurity problem: phishing succeeds when a message appears familiar, urgent, or financially important. Awareness platforms help users practice recognition in a controlled environment before they encounter real attacks.

Review area	Relevance to Aegis AI
Phishing awareness	The platform teaches users to inspect sender identity, links, urgency, and requests for sensitive information.
Simulation-based learning	Users learn by making decisions on realistic messages and immediately receiving feedback.
Adaptive training	Departments and individual sectors receive scenarios aligned with their daily risk patterns.
Gamification	Points and badges encourage repeated practice instead of one-time reading.
Inclusive security education	Senior awareness mode simplifies language and focuses on common scams.

3.1 Feedback, Suggested Improvements, and Grading Criteria

Evaluation category	Evidence in this documentation
Documentation quality	Complete report with planning, requirements, design, implementation, testing, and user manual.
Implementation quality	Structured project folders, modular backend routers, data models, AI service, and database schema.
Testing	Automated API tests and a documented manual test plan.
Presentation readiness	Demo flow, feature overview, expected outcomes, and final-report structure.
Suggested improvements	Authentication hardening, richer analytics, campaign scheduling, and monitoring.

4. Requirements Gathering

4.1 Stakeholder Analysis

Stakeholder	Needs
Employees	Short, realistic training that explains why a message is risky and what action is safe.
Security or IT team	Department-level risk visibility and repeatable awareness sessions.
Organization management	High-level dashboard showing progress, exposure, and training outcomes.
Individual learners	Personal practice, leaderboard motivation, and visible improvement.
Senior citizens	Simple language, common-scam examples, and quiz-based reinforcement.
Instructor/evaluator	Clear documentation, a runnable project, API docs, and a test plan.

4.2 User Stories and Use Cases

User story	Acceptance criteria
As an employee, I want to classify a suspicious message so I can practice identifying phishing.	The system displays a scenario, accepts a phishing/safe selection, and shows feedback.
As a manager, I want department risk results so I can identify where training is needed.	Dashboard displays attempts, average score, and risk level by department.
As an individual learner, I want to earn points so I stay motivated to practice.	Correct actions increase points and update the leaderboard and badge.
As a senior learner, I want simple security guidance so I can avoid common scams.	Checklist and quiz cover OTPs, unknown links, and account protection.
As a developer, I want API docs so I can test and integrate endpoints.	Swagger UI and written API documentation describe requests and responses.

4.3 Functional Requirements

ID	Requirement
FR-01	Generate cybersecurity training scenarios using the Claude API by department and sector.
FR-02	Support email, WhatsApp, and login-page scenario channels.
FR-03	Allow users to classify scenarios as phishing or safe.
FR-04	Score classifications and return feedback to the learner.
FR-05	Track organization departments, attempts, scores, correct answers, and risk levels.
FR-06	Provide individual training, points, badges, and leaderboard ranking.
FR-07	Provide a senior citizen awareness checklist and quiz.
FR-08	Expose REST API endpoints for scenarios, users, organizations, and leaderboard.
FR-09	Provide setup, deployment, API, and project documentation.

4.4 Non-functional Requirements

Category	Requirement
Performance	Interactive flows should respond quickly enough for classroom or live demonstration use.
Security	Secrets must be stored in environment variables; database access should use Supabase RLS and validated inputs.
Usability	Interface should be responsive, readable, and accessible to non-technical users.
Reliability	Backend should expose health checks and clear error responses.
Maintainability	Code should remain modular across frontend, API, models, services, and core config.
Scalability	Deployment should support cloud hosting and later additions such as caching or monitoring.
Portability	Project should run locally on Python 3.10+ with a modern browser and documented setup steps.

5. System Analysis and Design

5.1 Problem Statement and Objectives

The system solves the gap between passive cybersecurity advice and practical recognition skill. Instead of only listing phishing warning signs, Aegis AI presents scenario-based decisions that require users to inspect message context, sender identity, links, and requested actions.

Objective	Design response
Make training realistic	Use AI-generated content across multiple communication channels.
Make training role-specific	Separate organization departments and individual sectors.
Make improvement measurable	Track attempts, scores, risk levels, points, and badges.
Make learning inclusive	Add a senior-friendly awareness path with simple explanations.
Make the system demonstrable	Provide local run instructions, API docs, and database schema.

5.2 Use Case Overview

Actor	Use cases
Employee / learner	Generate scenario, classify message, view feedback, earn points.
Department manager	View department training statistics, compare risk levels, prioritize retraining.
Senior learner/helper	Read checklist, answer quiz, reinforce safe behavior.
Administrator	Configure environment variables, deploy frontend/backend, monitor health.
External services	Claude generates scenario content; Supabase stores relational project data and handles auth.

5.3 Software Architecture

The architecture follows a layered web application style: browser-based frontend, FastAPI backend, service integration layer, Supabase database and auth, and the Anthropic Claude AI provider.

User Browser → Frontend (HTML/CSS/JavaScript) → FastAPI routers → Pydantic request/response models → AI service → Anthropic Claude API → Supabase client → PostgreSQL tables (users, departments, scenarios).

Component	Responsibility
Frontend	Provides navigation, training screens, dashboard, and API calls.
FastAPI app	Routes scenario, organization, leaderboard, root, and health requests.
Models (Pydantic)	Validate scenario, organization, user, and leaderboard request/response shapes.
Services	Builds the scenario-generation prompt and parses the Claude response into application models.
Core	Holds application configuration and the Supabase client setup.
Supabase	Stores users, departments, and scenarios with Supabase Auth for identity.

5.4 Database Design and Data Modeling

The data model is relational and centered on users, departments, and training scenarios. UUID primary keys support cloud-hosted multi-user usage.

Entity	Key fields	Purpose
users	id, name, email, points, badge, created_at	Stores learner identity and gamification progress.
departments	id, key, name, attempts, score_sum, correct, risk_level, updated_at	Stores organization training categories and aggregated risk data.
scenarios	id, user_id, department_key, channel, is_phishing, user_classification, score, created_at	Stores each training attempt and its outcome.

ERD summary:

users.id 1 -> many scenarios.user_id

departments.key 1 -> many scenarios.department_key

Each scenario row records the classification decision, resulting score, and whether the scenario was truly phishing, which lets department statistics be recalculated from raw attempts.

5.5 Data Flow and System Behavior

Context-level data flow:

Learner -> Frontend -> FastAPI API -> AI Service -> Anthropic Claude API

Learner -> Frontend -> FastAPI API -> Supabase -> Training data / leaderboard / department stats

FastAPI API -> Learner: scenario content, classification result, dashboard data, leaderboard data

Flow	Steps
Scenario generation sequence	Learner selects department/sector and difficulty; frontend sends request; API validates input; AI service generates a structured scenario; response is displayed.
Classification activity	Learner reviews the scenario, chooses phishing or safe; system compares against the true label and calculates a score.
Dashboard update	Each training result updates department attempts, score sum, and correct count, which drive the risk level.
Senior awareness flow	Learner reviews the safety checklist, answers the quiz, and the result reinforces safe behavior.
Scenario state model	Not started -> generating -> ready for classification -> feedback shown -> completed.

5.6 Class and Component Design

Model / module	Responsibility
ScenarioRequest	Accepts department/sector and difficulty inputs.
ScenarioResponse	Returns channel, sender, message content, is_phishing flag, and explanation.
ClassificationRequest	Accepts the user's phishing/safe classification for a given scenario.
ClassificationResult	Returns correctness, score, and feedback.
OrganizationStats	Represents organization-level attempts, departments, and risk summary.
DepartmentUpdate	Represents score and correctness updates for a department.
User	Represents learner identity, points, and badge.
LeaderboardEntry	Represents ranking, name, points, and badge.

5.7 UI/UX Design and Prototyping

The frontend uses a clear, card-based navigation system for choosing a training path, a simulation panel for scenario review, a dashboard for department risk, and a leaderboard view. The design favors scanability and quick decisions during training.

Screen	Primary purpose
Home	Introduce Aegis AI and route users into organization or individual training.
Organization selection	Choose a department and view initial department risk status.
Simulation	Generate a scenario, classify it, and review feedback.
Dashboard	Display attempts, average score, and risk level per department.
Individual training	Practice by sector while earning points.
Senior awareness	Teach safety basics and complete a short awareness quiz.
Leaderboard	Show rankings, points, and badge progress.

5.8 Deployment and Integration Design

Deployment view:

User device -> Frontend host -> Backend host (FastAPI/Uvicorn) -> Supabase PostgreSQL + Auth -> Anthropic Claude API

Layer	Technology
Frontend	HTML5, CSS3, vanilla JavaScript
Backend	Python 3.10+, FastAPI, Uvicorn
Database	Supabase PostgreSQL with Supabase Auth and RLS
AI	Anthropic Claude API
Testing	pytest and FastAPI TestClient

6. Implementation and Execution

6.1 Source Code Organization

Path	Purpose
frontend/index.html	Single-page user interface for all training and dashboard flows.
backend/app/api/	API endpoint routers for scenarios, organizations, users, and leaderboard.
backend/app/models/	Pydantic models used by API requests and responses.
backend/app/services/	Business logic, including the Claude AI scenario-generation service.
backend/app/core/	Configuration and the Supabase database client.
backend/app/main.py	FastAPI application entry point.
backend/requirements.txt	Backend Python dependencies.
backend/.env.example	Template for required environment variables.
README.md	Project overview, setup, and quick-start guide.

6.2 Coding Standards, Modularity, and Error Handling

Practice	How the project applies it
Modular structure	API routers, models, services, and core settings are separated by responsibility.
Naming conventions	Files, functions, and API paths use meaningful names tied to their feature area.
Validation	Pydantic models define expected request and response structures.
Configuration	Environment variables isolate API keys and deployment-specific settings.
Security hygiene	Secrets are kept out of source control; Supabase RLS policies protect stored data.
Error handling	API errors are returned as structured responses with clear status codes.

6.3 System Requirements

Requirement	Details
Operating system	Windows, macOS, or Linux for local development.
Runtime	Python 3.10 or higher.
Browser	Modern browser supporting ES6 JavaScript.
Backend packages	fastapi, uvicorn, pydantic, supabase, anthropic, python-dotenv, and related libraries from requirements.txt.
External accounts	Supabase project and Anthropic API key.
Ports	Backend on 8000; frontend static server commonly on 8080.

6.4 Local Execution Guide

Backend setup:

```
cd backend  
  
python -m venv venv  
venv\Scripts\activate (Windows) or source venv/bin/activate (Mac/Linux)  
pip install -r requirements.txt  
cp .env.example .env  
uvicorn app.main:app --reload
```

Frontend setup:

```
cd frontend  
python -m http.server 8080
```

Environment variables:

```
SUPABASE_URL=your_supabase_project_url  
SUPABASE_KEY=your_supabase_anon_key  
ANTHROPIC_API_KEY=your_anthropic_api_key  
SECRET_KEY=your-secret-key-for-jwt  
ENVIRONMENT=development
```

6.5 Version Control and Collaboration

Area	Recommended practice
Repository	Host the project on GitHub with all documentation and source files.
Branching	Use feature branches for API, frontend, database, and documentation changes.
Commit messages	Use clear messages describing the feature, fix, or documentation update.
Pull requests	Include scope, screenshots or test evidence, and reviewer notes.
CI/CD	Add automated tests and deployment steps as the project moves toward production.

7. API Documentation

Base local API URL: <http://localhost:8000>. Interactive Swagger documentation is available at <http://localhost:8000/docs>.

Method	Endpoint	Purpose
GET	/	Return API info.
GET	/health	Health check.
POST	/api/scenarios/generate	Generate an AI scenario.
POST	/api/scenarios/classify	Submit a classification.
GET	/api/organizations/stats	Get organization stats.
POST	/api/organizations/departments/update	Update a department.
GET	/api/leaderboard	Get top users.
POST	/api/users	Create a user.
GET	/api/users/{id}	Get user info.

7.1 Representative Request and Response Shapes

Operation	Request / response summary
Generate scenario request	department, sector, and difficulty.
Generate scenario response	channel, sender details, message content, is_phishing, and explanation.
Classify request	scenario_id and user_classification.
Classification response	correctness, score, and feedback.
Department update	department_key, score, and correct.
Leaderboard response	Ranked entries containing name, points, and badge.

8. Testing and Quality Assurance

Testing focuses on API availability, response shapes, scenario generation, organization statistics, leaderboard output, and end-user workflows.

Test case	Expected result
Root endpoint	GET / returns status 200 and API metadata.
Health endpoint	GET /health returns status 200 and a healthy status.
Scenario generation	Scenario request returns structured content or a clear provider/configuration error.
Leaderboard	Leaderboard endpoint returns a non-empty ranked list.
Organization stats	Stats endpoint returns total attempts and average score fields.
Frontend training flow	User can start a scenario, classify it, and receive feedback.
Deployment smoke test	Production backend health check and frontend loading both succeed.

8.1 Automated Testing

```
cd backend
```

```
pip install -r requirements.txt
```

```
pytest tests/
```

8.2 Bug Report and Resolution Process

Field	Description
Bug ID	Unique identifier such as BUG-001.
Severity	Low, medium, high, or critical.
Area	Frontend, backend, database, AI service, or documentation.
Steps to reproduce	Exact steps needed to see the issue.
Expected result	What should happen.
Actual result	What happened instead.
Resolution	Fix summary, commit reference, and retest evidence.

9. User Manual

9.1 Organization Training

Step	Action
1	Open the frontend application.
2	Choose the Organizations path.
3	Select a department such as Accounting, HR, IT, Marketing, or Sales.
4	Generate a training scenario.
5	Read the message and classify it as phishing or safe.
6	Review the score and explanation.
7	Open the dashboard to compare department risk levels.

9.2 Individual and Senior Awareness Training

Mode	How to use it
Individual training	Choose a sector such as banking, shopping, or social media; generate scenarios; classify them; and collect points.
Leaderboard	Review ranking, point totals, and badge progress after completing training attempts.
Senior awareness	Review the checklist covering unknown links, OTP sharing, pressure tactics, and account protection.

10. Future Enhancements

Enhancement	Benefit
Authentication and roles	Support secure, role-based accounts for learners, managers, and administrators.
Advanced analytics	Trend lines, department comparisons, and exportable reports.
Scheduled campaigns	Allow organizations to launch recurring awareness exercises.
Certificates	Generate proof of completion for learners and departments.
Notifications	Send reminders and training invitations by email.
Multi-language support	Support Arabic and additional languages for broader accessibility.
Monitoring and logging	Track errors, API usage, latency, and production health.
Rate limiting and caching	Improve resilience, cost control, and performance at scale.

Appendix A. Project File Map

Folder / file	Documentation relevance
README.md	Main overview, feature list, stack, endpoints, and quick start.
frontend/index.html	Frontend UI for all training and dashboard flows.
backend/app/main.py	FastAPI application setup and entry point.
backend/app/api/	Scenario, organization, user, and leaderboard routers.
backend/app/models/	Pydantic models used across the API.
backend/app/services/	AI scenario-generation and business logic.
backend/app/core/	Configuration and Supabase client setup.
backend/requirements.txt	Backend Python dependencies.
backend/.env.example	Environment variable template.

Appendix B. Guideline Compliance Matrix

Guideline item	Where addressed
Project Proposal	Sections 1 and 2.1.
Project Plan	Section 2.2.
Task Assignment and Roles	Section 2.3.
Risk Assessment and KPIs	Sections 2.4 and 2.5.
Literature Review / Evaluation	Section 3.
Stakeholders, User Stories, Requirements	Section 4.
System Analysis and Design	Section 5.
Database Design and Data Modeling	Section 5.4.
Data Flow and Behavior	Section 5.5.
UI/UX Design	Section 5.7.
Deployment and Integration	Section 5.8.
Source Code and Execution	Section 6.
API Documentation	Section 7.
Testing and Quality Assurance	Section 8.
User Manual	Section 9.